

OPIS PROJEKTU

Projekt zakłada stworzenie dwuwymiarowej gry zręcznościowej, inspirowanej mechaniką popularnych tytułów takich jak Crossy Road czy Frogger. Aplikacja powstanie w języku C++ z wykorzystaniem środowiska Qt Creator. Głównym zadaniem w grze jest bezpieczne przeprowadzenie postaci (żaby) przez ruchliwą ulicę w taki sposób, aby uniknąć kolizji z nadjeżdżającymi pojazdami. Utrudnieniem na trasie będą trzy różne typy samochodów, charakteryzujące się odmienną grafiką.

Wyróżnikiem projektu są dwa główne tryby działania, które można wybrać z poziomu menu głównego:

Tryb "Sterowany przez gracza": Przeznaczony do klasycznej rozgrywki. Użytkownik przejmuje pełną kontrolę nad postacią, sterując nią za pomocą strzałek na klawiaturze. Ten tryb opiera się na refleksie i zręczności gracza.

Tryb "Trening AI": Moduł eksperymentalny, w którym kontrolę nad bohaterem oddajemy algorytmowi sztucznej inteligencji. Zadaniem tego trybu jest obserwacja procesu uczenia się maszyny – z każdą kolejną próbą algorytm analizuje błędy, by ostatecznie samodzielnie i bezkolizyjnie pokonać całą szerokość ulicy.

Podsumowując, aplikacja to połączenie w pełni grywalnej produkcji zręcznościowej z praktyczną wizualizacją tego, jak algorytmy sztucznej inteligencji potrafią adaptować się do dynamicznego środowiska i uczyć się unikania przeszkód.

PLAN PRACY NAD PROJEKTEM

[Tydzień 1 / Tydzień 10 zajęć] – Inicjalizacja i Repozytorium

- Konfiguracja środowiska
- Tworzenie repozytorium Git i struktury plików projektu
- Przygotowanie nagłówek głównych klas sterujących

[Tydzień 2 / Tydzień 11 zajęć] – ETAP 0: Wymagania i Makieta GUI

- Spisanie wymagań funkcjonalnych oraz założeń projektowych gry.
- Opracowanie makiet interfejsu
- Powiązanie na diagramie: Zręby struktur pod GameWindow oraz MenuScreen

[Tydzień 3 / Tydzień 12 zajęć] – ETAP 1.a: Architektura i Sytuacje Wyjątkowe

- Przeniesienie diagramu klas do oficjalnej dokumentacji projektu.
- Analiza i specyfikacja obsługi sytuacji wyjątkowych - Powiązanie na diagramie: Zdefiniowanie interfejsów klas abstrakcyjnych Car i PlayerSkin

[Tydzień 4 / Tydzień 13 zajęć] – ETAP 1: Szczegóły Architektury i Plan Testów

- Uszczegółowienie powiązań (agregacja pasów ruchu Lane i wykrywacza kolizji CollisionDetector w klasie Game).
- Opracowanie dokumentu "Plan Testów" (scenariusze testowe dla ruchu, punktacji i wykrywania kolizji).
- Powiązanie na diagramie: Przygotowanie szablonów testów dla CollisionDetector::checkCollision() i ScoreManager.

[Tydzień 5 / Tydzień 14 zajęć] – ETAP 2.a: Główna Implementacja i Integracja (Zdalne - cz. puli 25 pkt)

- Implementacja rdzenia logiki: ruch gracza , generowanie pojazdów oraz działanie klas SUV, Truck, SmallCar.
- Wdrożenie CollisionDetector oraz systemów pobocznych: AudioManager (dźwięki), ScoreManager i SkinManager
- Połączenie logiki z interfejsem graficznym i grafikami – **cel: na koniec tygodnia gra w pełni działa jako kompletna całość**.

[Tydzień 6 / Tydzień 15 zajęć] – ETAP 2: Bufor, Poprawki, Testy i Obrona

- **Bufor bezpieczeństwa:** Gra już działa – ten czas przeznaczony jest na ostateczne polerowanie kodu i usuwanie znalezionych błędów (bugfixy).
- Przeprowadzenie oficjalnych testów z przygotowanego wcześniej planu testów.
- Aktualizacja diagramu klas do stanu fakty

Podział pracy:

Bartosz Moszczyński: Logika samochodów, przegrana

Jan Sotrel: Implementacja i nauka Ai, logika zdobywania punktów

Stanisław Wożakowski: Logika gracza, postaci, poruszania się